

## Project Executable Files & Execution Guide

|               |                                                             |
|---------------|-------------------------------------------------------------|
| Date          |                                                             |
| Team ID       |                                                             |
| Project Name  | OptiCrop: Smart Agricultural Production Optimization Engine |
| Maximum Marks | 3 Marks                                                     |

### Project Executable Files & Execution Guide:

To successfully evaluate our project, evaluators require access to our source code, live deployments, and clear instructions on how to build and run our application locally. Completed the sections below to document our project repositories and execution steps.

### Step 1: Project Links & Access

| Resource Type | URL / Access Link |
|---------------|-------------------|
|               |                   |

|                                                          |                                                                                                                                                                                           |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source Code Repository<br>(e.g., GitHub, GitLab)         | <a href="https://github.com/Kanthisriamarthala/OptiCrop">https://github.com/Kanthisriamarthala/OptiCrop</a>                                                                               |
| Live Deployed Application<br>(e.g., Vercel, AWS, Heroku) | <a href="https://opti-crop-nine.vercel.app/">https://opti-crop-nine.vercel.app/</a>                                                                                                       |
| Demo Video Link<br>(Optional but recommended)            | <a href="https://drive.google.com/file/d/1oMuWCZriScKutjqzOash4vKEluSsBVIP/view?usp=drive_link">https://drive.google.com/file/d/1oMuWCZriScKutjqzOash4vKEluSsBVIP/view?usp=drive_link</a> |

## Step 2: Key Executable Files & Folder Structure

Briefly explained the primary directories and where the core execution files are located within our repository.

| S.No | Folder / File Name            | Description & Purpose                                                                                                                                                                                                                             |
|------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | /backend/app.py               | Main entry point for the Flask REST API server. Defines all 8 API endpoints (/api/health, /api/predict, /api/suitability, /api/dashboard, /api/metrics, /api/cropinfo, /api/images, /api/crops). Initializes Flask-CORS and loads ML model.       |
| 2    | /backend/model/predict.py     | Core prediction engine. Contains predict_crop() and assess_suitability() functions. Loads trained Random Forest model and label encoder to predict based on 7 input features.                                                                     |
| 3    | /backend/model/train_model.py | Model training pipeline. Trains and evaluates 6 ML classifiers (Random Forest, SVM, Decision Tree, Logistic Regression, KNN, Naive Bayes) with cross-validation. Also runs K-Means clustering with PCA. Saves all artifacts (.pkl, metrics.json). |

|   |                                 |                                                                                                                                                                                 |
|---|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | /backend/utils/visualization.py | Generates all 10 chart images (crop distribution, NPK, temperature, humidity, rainfall, pH, correlation heatmap, boxplots, pairplot, K-Means clusters). Called on server start. |
| 5 | /backend/utils/preprocessing.py | Loads and preprocesses Crop_recommendation.csv. Returns clean DataFrame for training and dashboard statistics computation.                                                      |
| 6 | /backend/config.py              | Central configuration. Contains CROP_INFO dictionary (22 crops with description, ideal temp/rainfall/pH, tips), file paths, and server settings.                                |
| 7 | /backend/requirements.txt       | Python dependency list (Flask, Flask-CORS, scikit-learn, pandas, numpy, matplotlib, seaborn, scipy, joblib, gunicorn).                                                          |
| 8 | /frontend/recommend.html        | Crop recommendation UI. Form collects 7 parameters, sends POST to /api/predict, renders result card with crop name, confidence %, and detailed crop info.                       |

|    |                          |                                                                                                                                                                          |
|----|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9  | /frontend/dashboard.html | Research dashboard. Fetches /api/dashboard, renders model comparison table, Chart.js bar chart, dataset statistics, correlation insights, K-Means card, 10 chart images. |
| 10 | /frontend/js/script.js   | Client-side JavaScript. Defines API_BASE global, form validation, contact form handler, tooltips, alert autodismiss, smooth scroll.                                      |

| Step | Action / Terminal Command | Expected Outcome / Notes |
|------|---------------------------|--------------------------|
|------|---------------------------|--------------------------|

|          |                                                                        |                                            |
|----------|------------------------------------------------------------------------|--------------------------------------------|
| <b>1</b> | <code>git clone https://github.com/[your-username]/OptiCrop.git</code> | Downloads the project to the local machine |
|----------|------------------------------------------------------------------------|--------------------------------------------|

### Step 3: Local Setup & Execution Instructions

Provided a step-by-step guide to installing dependencies and launching the application on a local machine.

|          |                                                                                                                   |                                                                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>3</b> | <code>cd backend python app.py</code>                                                                             | Launches the Flask backend API server on <code>http://localhost:5000</code><br>Autogenerates 10 chart images on first run.                                |
| <b>4</b> | Verify backend is running:<br><code>curl http://localhost:5000/api/health</code>                                  | Expected response:<br><code>{"status":"ok","model_ready":true}</code>                                                                                     |
| <b>5</b> | Open <code>frontend/index.html</code> in browser<br>(or use VS Code Live Server on <code>frontend/</code> folder) | Opens the OptiCrop homepage. Navigate to Recommend, Suitability, or Dashboard.                                                                            |
| <b>6</b> | Open <code>frontend/recommend.html</code><br>Fill in N, P, K, temp, humidity, pH, rainfall<br>Click Predict Crop  | Frontend calls <code>http://localhost:5000/api/predict</code><br>Displays predicted crop, confidence %, description, ideal conditions, tips.              |
| <b>7</b> | Open <code>frontend/suitability.html</code><br>Enter parameters and click Analyze Suitability                     | Frontend calls <code>http://localhost:5000/api/suitability</code><br>Shows suitability status, productivity level, environmental badges, recommendations. |

|    |                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                               |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8  | Open frontend/dashboard.html                                                                                                                                                                                                                                      | <p>Automatically fetches /api/dashboard and renders:</p> <ul style="list-style-type: none"> <li>- Summary cards &amp; model comparison table</li> <li>- Chart.js bar chart (accuracy/precision/recall/F1)</li> <li>- Dataset statistics &amp; correlation insights</li> <li>- K-Means cluster analysis card</li> <li>- 10 chart images (via API or local fallback)</li> </ul> |
| 9  | <p>(Optional) Test prediction via curl: curl -X POST http://localhost:5000/api/predict ^</p> <p>-H "Content-Type: application/json" ^</p> <p>-d</p> <p>"{"nitrogen":90,"phosphorous":40,"potassium":40,"temperature":20,"humidity":80,"ph":7,"rainfall":100}"</p> | <p>Expected response:</p> <pre>{"crop":"rice","confidence":0.28,"crop_info":{"..."}}</pre>                                                                                                                                                                                                                                                                                    |
| 10 | <p>Deploy frontend to Vercel: cd frontend &amp;&amp; vercel</p> <p>Deploy backend to Render: Push backend/ folder to GitHub Set start command: gunicorn app:app --bind 0.0.0.0:\$PORT</p> <p>Set FRONTEND_URL env var to Vercel URL</p>                           | <p>Frontend: https://opticropks4.vercel.app/</p> <p>Backend: Render (auto HTTPS, auto-restart)</p>                                                                                                                                                                                                                                                                            |